

Practical 02

Your First Honest Model

Split the data, train a model,
and prove it beats guessing

Student	Lakhan Gupta
Enrolment number	S24CSEU0067
Programme	B.Tech CSE · 5th Semester
Institution	Bennett University · Session 2026-27
Course outcome	CO2 · follows lecture L03
Notebook	P02_S24CSEU0067.ipynb
Self-check result	9 of 9 checks passed

SOURCE CODE REPOSITORY

https://github.com/LakhanxGupta/MLOPS_BU

https://github.com/LakhanxGupta/MLOPS_BU/tree/main/P02_lab

01 Step 0 - Check the workbench

Preflight: confirms the interpreter and that numpy, pandas and scikit-learn are importable.

```
In [1]: # Step 0 --- check the workbench before we start.  
# This cell only looks; it changes nothing. Run it and read the last line.
```

```
import sys  
from pathlib import Path  
  
print("Python :", sys.version.split()[0])  
print("Folder :", Path.cwd().name)  
  
_missing = []  
for _name in ['numpy', 'pandas', 'sklearn']:  
    try:  
        __import__(_name)  
    except ImportError:  
        _missing.append(_name)  
  
for _name in ['numpy', 'pandas', 'sklearn']:  
    _mark = "missing" if _name in _missing else "ok"  
    print(f"  {_name:<14} {_mark}")  
  
if _missing:  
    print()  
    print("STOP. Some libraries are missing:", ", ".join(_missing))  
    print("Ask your instructor to run the setup in labs/SETUP.md.")  
else:  
    print()  
    print("All good. You can carry on to Step 1.")
```

```
Python : 3.13.14  
Folder : P02-first-model  
numpy      ok  
pandas     ok  
sklearn    ok
```

All good. You can carry on to Step 1.

02 Step 0b - The dataset

Confirms the shared 600-row delivery dataset is present; rebuilds it from seed 42 only if missing.

```
In [2]: # The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path(".") / "data" / "delivery_times.csv"

def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[i]])
    return path

if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found ->", DATA)

dataset found -> ../data/delivery_times.csv
```

03 Step 1 - Load the deliveries and look at them

Reads the CSV and prints its shape and first five rows before any modelling is attempted.

```
In [3]: import numpy as np
import pandas as pd

orders = pd.read_csv(DATA)

print("rows, columns:", orders.shape)
print()
print(orders.head())

rows, columns: (600, 5)

   distance_km  prep_time_min  traffic_level  rain  delivery_min
0         9.40             17              1     0          51.3
1         5.55             24              2     1          54.2
2        10.37             28              3     0          67.7
3         8.52             23              2     0          51.2
4         1.58             29              2     1          42.1
```

04 Step 2 - Separate the features from the target

Splits the table into X, the four known facts about an order, and y, the delivery time to predict.

```
In [4]: FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
TARGET = "delivery_min"

X = orders[FEATURES]
y = orders[TARGET]

print("X shape:", X.shape, " <- 600 orders, 4 features each")
print("y shape:", y.shape, " <- 600 answers")
print()
print(X.head(3))
print()
print(y.head(3))

X shape: (600, 4) <- 600 orders, 4 features each
y shape: (600,) <- 600 answers

   distance_km  prep_time_min  traffic_level  rain
0         9.40             17              1     0
1         5.55             24              2     1
2        10.37             28              3     0

0    51.3
1    54.2
2    67.7
Name: delivery_min, dtype: float64
```

05 Step 3 - Cut the data in two

An 80/20 train-test split with random_state=42, so every student holds back the same 120 orders.

```
In [5]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("training on:", len(X_train), "orders")
print("testing on :", len(X_test), "orders")
print("total      :", len(X_train) + len(X_test))

training on: 480 orders
testing on : 120 orders
total      : 600
```

06 Step 4 - The number to beat: always guess the average

The baseline predictor answers 47.1 minutes for every order and misses by 10.32 minutes on average.

```
In [6]: from sklearn.metrics import mean_absolute_error, mean_squared_error

average_time = y_train.mean()
print(f"average delivery time in training data: {average_time:.1f} min")

baseline_guesses = np.full(len(y_test), average_time)
baseline_mae = mean_absolute_error(y_test, baseline_guesses)

print(f"BASELINE MAE: {baseline_mae:.2f} minutes")
print()
print("Meaning: guessing the average is wrong by about")
print(f"{baseline_mae:.0f} minutes on a typical order.")

average delivery time in training data: 47.1 min
BASELINE MAE: 10.32 minutes

Meaning: guessing the average is wrong by about
10 minutes on a typical order.
```

07 Step 5 - Train a real model

LinearRegression fitted on the 480 training orders, learning one coefficient per feature.

```
In [7]: from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print("Model trained on", len(X_train), "orders.")
print("It has learned", len(model.coef_), "numbers, one per feature.")
```

Model trained on 480 orders.
It has learned 4 numbers, one per feature.

08 Step 6 - Score it on the data it has never seen

On the held-back test set the model scores MAE 1.92 and RMSE 2.48 minutes, 81% better than the baseline.

```
In [9]: predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
rmse = float(np.sqrt(mean_squared_error(y_test, predictions)))

print(f"BASELINE MAE : {baseline_mae:6.2f} minutes")
print(f"MODEL MAE : {mae:6.2f} minutes")
print(f"MODEL RMSE : {rmse:6.2f} minutes")
print()
improvement = 100 * (baseline_mae - mae) / baseline_mae
print(f"The model is {improvement:.0f}% better than guessing.")
```

BASELINE MAE : 10.32 minutes
MODEL MAE : 1.92 minutes
MODEL RMSE : 2.48 minutes

The model is 81% better than guessing.

09 Step 7 - Ask the model what it learned

The learned coefficients read back in plain units: minutes added per kilometre, per prep minute, per traffic level and for rain.

```
In [10]: learned = pd.DataFrame({
    "feature": FEATURES,
    "minutes added per unit": model.coef_.round(2),
})

print(learned.to_string(index=False))
print()
print(f"starting point (intercept): {model.intercept_: .1f} minutes")
```

feature	minutes added per unit
distance_km	3.07
prep_time_min	0.65
traffic_level	4.13
rain	5.55

starting point (intercept): 6.4 minutes

10 Step 8 - Predict one new order

A single unseen order in, one number out: 43.0 minutes for a 5 km delivery in medium traffic.

```
In [11]: new_order = pd.DataFrame([
    "distance_km": 5.0,
    "prep_time_min": 20,
    "traffic_level": 2,
    "rain": 0,
])

minutes = model.predict(new_order)[0]
print(f"Predicted delivery time: {minutes:.1f} minutes")
```

Predicted delivery time: 43.0 minutes

11 Task T1 - Work out the baseline yourself

Task T1. A median baseline scores MAE 10.33 against the mean baseline's 10.32, so the mean is the better guess.

```
In [12]: # TODO: predict the median for every test order, then measure the MAE.
median_time = y_train.median()
median_guesses = np.full(len(y_test), median_time)
T1_median_mae = mean_absolute_error(y_test, median_guesses)

# TODO: "mean" or "median" — which baseline was better?
T1_which_is_better = "mean" if baseline_mae < T1_median_mae else "median"

print("median baseline MAE:", T1_median_mae)
print("better baseline      :", T1_which_is_better)

median baseline MAE: 10.331666666666669
better baseline      : mean
```

12 Task T2 - Does the split change the answer?

Task T2. Retrained on a 70/30 split with seed 7: MAE 2.10 minutes, close to the original, so the result was not a lucky split.

```
In [13]: # TODO: split 70/30 with random_state=7
X_tr2, X_te2, y_tr2, y_te2 = train_test_split(
    X, y, test_size=0.3, random_state=7
)

# TODO: train a NEW model on the new training data
model2 = LinearRegression().fit(X_tr2, y_tr2)

# TODO: its MAE on the new test set
T2_mae = mean_absolute_error(y_te2, model2.predict(X_te2))

print("70/30 split, seed 7 -> MAE", T2_mae)

70/30 split, seed 7 -> MAE 2.0963348724086432
```

13 Task T3 - One order in, one number out

Task T3. `predict_minutes()` wraps the model; a rainy 3 km order returns 35.0 minutes.

```
In [14]: def predict_minutes(distance_km, prep_time_min, traffic_level, rain):
# TODO: build a one-row DataFrame, predict, round to 1 decimal.
    one_order = pd.DataFrame([
        "distance_km": distance_km,
        "prep_time_min": prep_time_min,
        "traffic_level": traffic_level,
        "rain": rain,
    ])
    return round(float(model.predict(one_order)[0]), 1)

# TODO: 3 km, 15 minutes prep, traffic level 1, raining
T3_rainy_order = predict_minutes(3.0, 15, 1, 1)

print("rainy 3 km order ->", T3_rainy_order, "minutes")

rainy 3 km order -> 35.0 minutes
```

14 Self-check - marking table

Self-check: 9 of 9 checks passed.

```
In [15]: # -----
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# -----

_results = []

def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))

_check('T1 | T1_median_mae is the MAE of a median baseline', lambda: abs(float(T1_median_mae) - mean_absolute_error(y_test,
np.full(len(y_test), y_train.median())) < 0.01))
_check('T1 | T1_which_is_better names the lower-MAE baseline', lambda: T1_which_is_better == ('mean' if baseline_mae <
mean_absolute_error(y_test, np.full(len(y_test), y_train.median())) else 'median'))
_check('T2 | the new test set holds 30% of the orders', lambda: len(X_te2) == 180)
_check('T2 | model2 is a trained LinearRegression', lambda: hasattr(model2, 'coef_') and len(model2.coef_) == 4)
_check('T2 | T2_mae is that model's MAE on the new test set', lambda: abs(float(T2_mae) - mean_absolute_error(y_te2,
model2.predict(X_te2))) < 0.01)
_check('T3 | predict_minutes returns a single number', lambda: isinstance(predict_minutes(5.0, 20, 2, 0), (int, float)))
_check('T3 | it agrees with the trained model', lambda: abs(predict_minutes(5.0, 20, 2, 0) -
float(model.predict(pd.DataFrame([{'distance_km': 5.0, 'prep_time_min': 20, 'traffic_level': 2, 'rain': 0}]))[0])) < 0.06)
_check('T3 | rain makes the same order take longer', lambda: predict_minutes(3.0, 15, 1, 1) > predict_minutes(3.0, 15, 1, 0))
_check('T3 | T3_rainy_order is the rainy 3 km prediction', lambda: abs(float(T3_rainy_order) - predict_minutes(3.0, 15, 1, 1)) < 0.06)

print("=====")
print("SELF-CHECK   Practical 02 --- Your First Honest Model")
print("=====")
for _label, _ok in _results:
    print(f" [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("-----")
_passed = sum(1 for _, _ok in _results if _ok)
print(f" {_passed} of {len(_results)} checks passed")
print("=====")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again.")

=====
SELF-CHECK   Practical 02 --- Your First Honest Model
=====
[PASS] T1 | T1_median_mae is the MAE of a median baseline
[PASS] T1 | T1_which_is_better names the lower-MAE baseline
[PASS] T2 | the new test set holds 30% of the orders
[PASS] T2 | model2 is a trained LinearRegression
[PASS] T2 | T2_mae is that model's MAE on the new test set
[PASS] T3 | predict_minutes returns a single number
[PASS] T3 | it agrees with the trained model
[PASS] T3 | rain makes the same order take longer
[PASS] T3 | T3_rainy_order is the rainy 3 km prediction
=====
9 of 9 checks passed
=====
Well done. Save the notebook and submit it.
```